

# Gestion des dépendances, risques et maintenabilité

18/09/26 - 28/09/26 - 07/10-26

# Qui suis-je ?

Philippe AUSSEL

- Début de codeur à 14 ans sur CPC 464
- 10 ans en tant que développeur
- 18 ans en tant qu'architecte/manager/CTO
- Mais toujours développeur dans l'âme



# Programme

- Jour 1 — Comprendre et maîtriser ses dépendances.
- Jour 2 — Découpler, packager et sécuriser ses choix.
- Jour 3 — Appliquer, puis évaluer.

# JOUR 1 — Comprendre et maîtriser ses dépendances.



# Partie 1 : C'est quoi une dépendance ?

# Définition

Une dépendance est tout élément dont un module a besoin pour fonctionner. Cela peut être une fonction, une classe, une bibliothèque, un service externe... Sans cette dépendance, le module ne peut pas exécuter sa tâche.

“A dépend de B” signifie : si B change, A peut casser.

# Des dépendances, il y en a partout

- Une librairie, un framework, une package
- Une API, un service cloud, ..
- Une base de données, un partage de fichier
- Un protocole de communication

# Dépendances internes

- Un module Front qui dépend d'un module Services
- Une classe qui dépend d'une autre classe
- Un projet partagé (Common, Shared...)

# Dépendances externes

- Librairies : NuGet, npm, pip...
- Frameworks
- API tierces
- Bases de données

Les externes sont souvent plus risquées : on les maîtrise moins.

# Directes ou transitives

## Directes

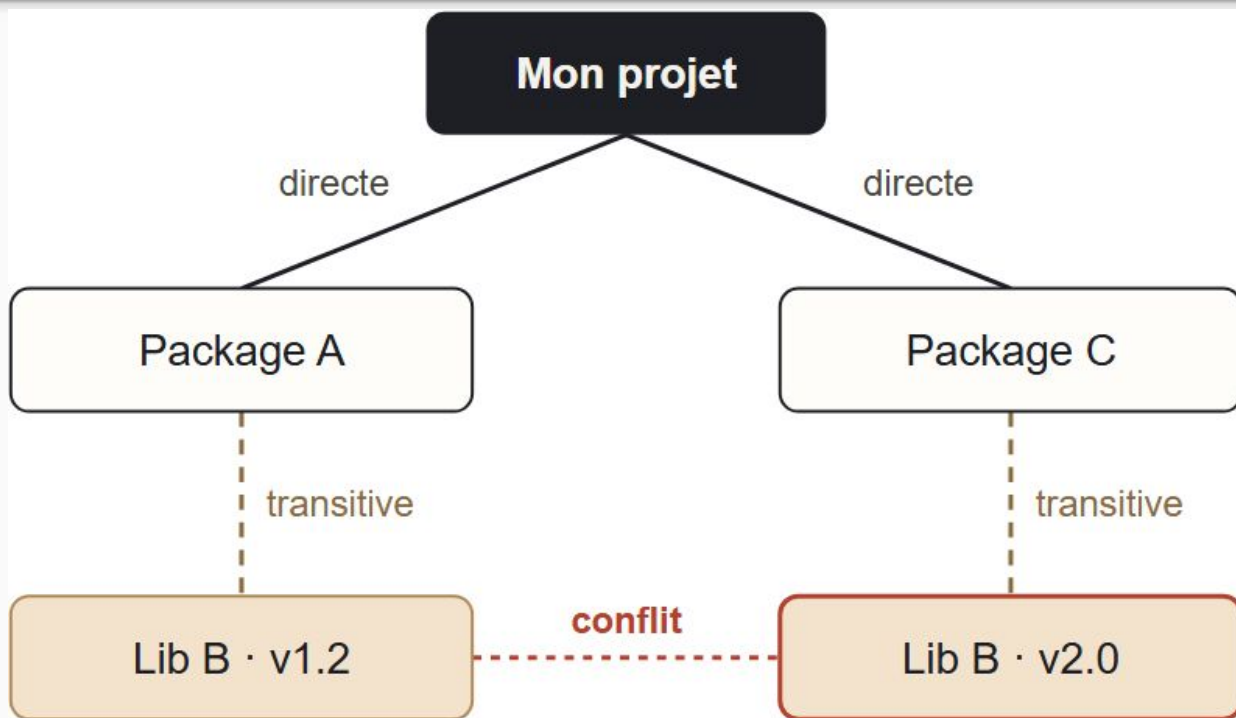
Vous l'appellez explicitement dans votre code.

## Transitives

Jamais mentionnée : elle arrive avec une autre. Votre projet utilise A, A utilise B, donc votre projet dépend aussi de B.

Invisibles, elles sont souvent sources de bugs.

# Directes ou transitives



# Explicites ou implicites

## **Explicites : Visibles dans le code**

- import, using, #include
- new MaClasse()
- Un appel d'API écrit en clair

## **Implicites : Rarement documentées**

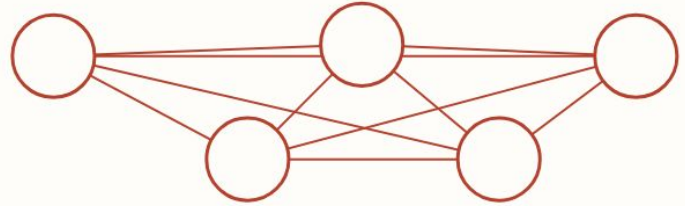
- Variables d'environnement
- Fichiers de configuration
- Format de données attendu

# Couplage



## **Couplage faible : c'est bon**

Un changement dans un module casse peu le reste.



## **Couplage fort : danger**

Effet domino, maintenance difficile.

L'objectif : minimiser le couplage, pas éliminer les dépendances.

# Cohésion

Faible cohésion : Un module qui fait tout et n'importe quoi.

```
class ClientManager
{
    bool ValiderSiret(string s);
    void EnvoyerEmail(Client c);
    byte[] FacturePdf(Facture f);
    void ExporterCsv(Client[] l);
}
```

Haute cohésion : Des modules clairs, stables, faciles à maintenir.

```
class SiretValidator { ... }

class ClientMailer { ... }

class InvoicePdfGenerator { ... }

class ClientCsvExporter { ... }
```

# Couplage faible + cohésion forte = architecture saine

|                 | Couplage faible                                                       | Couplage fort                                                      |
|-----------------|-----------------------------------------------------------------------|--------------------------------------------------------------------|
| Cohésion forte  | Architecture saine,<br>modules clairs,<br>changements<br>localisés.   | Modules rigides,<br>chacun est clair, mais<br>tout bouge ensemble. |
| Cohésion faible | Code éparpillé et<br>Indépendant, mais<br>responsabilités<br>diluées. | Tout dépend de tout,<br>rien n'est clair.                          |

# Pourquoi les dépendances fragilisent un projet

- **Propagation** : Si la dépendance casse, tout ce qui repose dessus casse aussi.
- **Mise à jour** : Une nouvelle version introduit une incompatibilité.
- **Disponibilité** : L'API externe est lente ou indisponible.
- **Sécurité** : Une vulnérabilité est découverte dans un package.
- **Humain** : Une seule personne comprend la dépendance.

Une dépendance mal gérée = dette technique + instabilité.

# Quand une dépendance devient-elle un problème ?

- **Elle change souvent** : API instable, ruptures de compatibilité fréquentes
- **Plus maintenue** : Ni correctifs de bugs, ni correctifs de sécurité
- **Fortement couplée** : Ses types et ses appels sont partout dans votre code
- **Utilisée partout** : Un SPOF en puissance
- **Non isolée** : Aucune interface entre elle et le métier
- **Licence risquée** : GPL ou AGPL dans un produit propriétaire

Le problème n'est pas la dépendance : c'est l'absence de contrôle.

# Une dépendance n'est pas que du code

- **Vendor lock-in (Fournisseur)** — Sortir coûte cher : formats fermés, services managés, compétences spécifiques
- **Pricing (Prix)** — Le modèle tarifaire ou la licence peuvent changer du jour au lendemain
- **Souveraineté (Juridiction)** — Où sont les données, et quel droit s'applique à l'opérateur ?
- **Bus factor (Personnes)** — Un mainteneur unique, une compétence rare dans l'équipe

Approfondi durant le jour 2

# Exercice

Inventoriez les dépendances d'un projet que vous connaissez

- Listez au moins 10 dépendances : code, infrastructure, services, configuration.
- Classez-les : interne ou externe, directe ou transitive, explicite ou implicite.
- Notez leur couplage avec votre code : faible, moyen ou fort.
- Entourez celle qui vous inquiète le plus, et dites pourquoi.

# A retenir

- Une dépendance est tout ce dont votre code a besoin, et pas seulement du code.
- Moins vous contrôlez une dépendance, plus elle est risquée.
- Transitives et implicites sont les plus dangereuses : on ne les voit pas.
- Visez un couplage faible et une cohésion forte.

# Partie 2 : Identifier les dépendances dans un projet



# Quatre endroits où chercher

- **Le code** : import, using, #include, require()
- **Le réseau** : appels HTTP, SDK cloud, files de messages
- **Les manifestes** : .csproj, package.json, requirements.txt
- **Les ressources** : bases, stockage, cache, secrets

Puis ce qui ne se voit nulle part : les dépendances cachées.

# Méthode 1 : Repérer les imports

**Comment faire** : Parcourir les fichiers source ou utiliser un outil d'analyse statique pour ces mots-clés.

**Ce que ça révèle** : Les dépendances directes et explicites de chaque fichier.

**Premier réflexe d'analyse** : lire la liste des imports.

```
// C#  
using MyProject.Data;  
using Newtonsoft.Json;  
  
// JavaScript  
import express from 'express';  
  
// Python  
from pandas import DataFrame  
  
// C / C++  
#include <curl/curl.h>
```

# Méthode 2 : Traquer les appels réseau

Ce qu'on cherche

- API REST et GraphQL
- SDK cloud (Azure, AWS...)
- Microservices internes et externes
- Files de messages : Kafka, RabbitMQ

Comment : Rechercher fetch, axios.get, HttpClient.SendAsync... ou analyser les logs de trafic sortant.

```
var url = "https://api.open-  
meteo.com/v1/forecast"  
+ $"?latitude={lat}&longitude={lon}";  
  
var response = await _http.GetAsync(url);  
response.EnsureSuccessStatusCode();
```

# Méthode 3 : Lire les manifestes

|            |                                     |                    |               |
|------------|-------------------------------------|--------------------|---------------|
| .NET       | *.csproj                            | packages.lock.json | NuGet         |
| JavaScript | package.json                        | package-lock.json  | npm, Yarn     |
| Python     | requirements.txt,<br>pyproject.toml | poetry.lock        | pip, Poetry   |
| Java       | pom.xml, build.gradle               |                    | Maven, Gradle |
| PHP        | composer.json                       | composer.lock      | Composer      |

# Méthode 4 : Suivre l'accès aux ressources

- Bases de données SQL et NoSQL
- Systèmes de fichiers
- Stockage objet : S3, Azure Blob
- Caches : Redis
- Secrets et identité : Key Vault, annuaire

Chercher les chaînes de connexion, les configurations d'accès et les drivers ou clients utilisés.

```
appsettings.json
{
  "ConnectionStrings": {
    "Crm": "Server=sql01;Database=Crm;..."
  },
  "Storage": {
    "Exports":
    "https://acme.blob.core.windows.net"
  },
  "Redis": "cache01:6379"
}
```

# Les dépendances cachées

- **Configuration et environnement** : `process.env.DB_HOST`, fichiers YAML ou JSON qui changent le comportement
- **Singletons et état global** : Une instance unique et globale : couplage fort, impossible à remplacer en test
- **Outils externes** : Le code lance un binaire (ffmpeg, gzip, un compilateur) qu'il ne gère pas
- **Contexte d'exécution** : Horloge, fuseau horaire, culture (séparateur décimal), système d'exploitation

Source majeure de dette technique et de régressions surprises.

# Combien de dépendances dans cette méthode ?

```
public void Export()
{
    var host = Environment.GetEnvironmentVariable("DB_HOST");
    var rows = Database.Instance.Query("SELECT * FROM Clients");
    var file = $"C:\\exports\\clients_{DateTime.Now:yyyyMMdd}.csv";
    File.WriteAllText(file, ToCsv(rows));
    Process.Start("gzip", file);
}
```

## Au moins 8

1. Variable d'environnement DB\_HOST
2. Singleton Database.Instance
3. Schéma de la table Clients
4. Chemin Windows C:\\exports
5. Horloge système
6. Système de fichiers
7. Binaire gzip installé
8. Culture utilisée par ToCsv

# Les outils pour lister et auditer

|                |                                                    |                                                                    |
|----------------|----------------------------------------------------|--------------------------------------------------------------------|
| .NET           | dotnet list package --include-transitive           | dotnet list package --vulnerable<br>dotnet list package --outdated |
| JavaScript     | npm ls --all                                       | npm audit<br>npm outdated                                          |
| Python         | pipdeptree                                         | pip-audit<br>pip list --outdated                                   |
| Graphe de code | NDepend (.NET) · dependency-cruiser,<br>madge (JS) | Visualiser le couplage et les cycles                               |

# Cartographier : de la liste au graphe

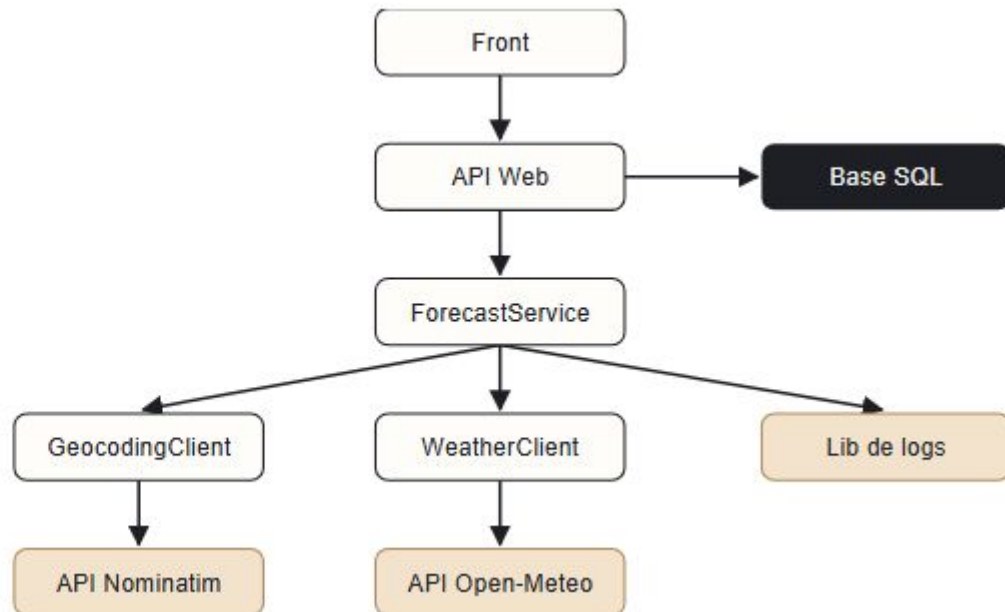
Un listing ne montre pas les interconnexions. Le graphe donne une vue **macro** (qui dépend de qui) et **micro** (quelles classes).

☐ Module interne

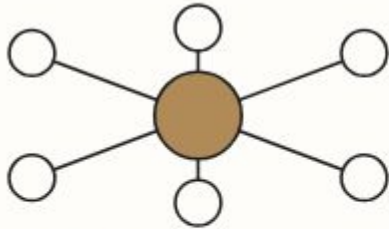
☒ Externe : package ou API

☒ Infrastructure

— « dépend de »

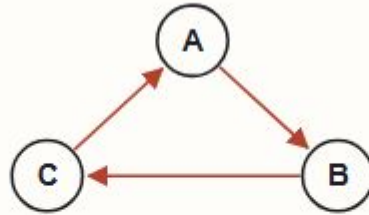


# Ce que le graphe révèle



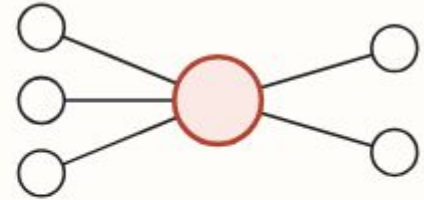
## Couplage fort

Un nœud relié à beaucoup d'autres :  
chaque changement s'y propage.



## Dépendances circulaires

$A \rightarrow B \rightarrow C \rightarrow A$  : difficile à tester et à décomposer.



## Chemins critiques et SPOF

Le point par où transite la majorité  
des flux.

# Détecter les dépendances critiques

Les 3 critères :

- **Fréquence** : Combien de modules l'utilisent ?
- **Rôle métier** : Est-elle indispensable au cœur de métier ?
- **Contrôle** : Pouvez-vous la corriger ou la remplacer ?

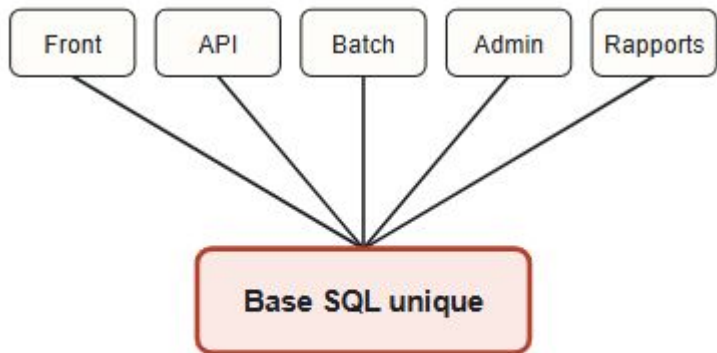
# SPOF : Single Point of Failure

## Exemples classiques

- Une base de données unique
- Un service d'authentification centralisé
- Une librairie de logs utilisée par 100 % des modules

## Stratégies

- Isoler derrière une interface
- Dédoubler : réplication, failover
- Circuit breaker et cache pour un mode dégradé



**Elle tombe : tout s'arrête**

# A retenir

- Imports et manifestes révèlent les dépendances directes et les librairies externes.
- Appels réseau et ressources montrent ce que vous ne contrôlez pas..
- Configuration, singletons et contexte d'exécution cachent des dépendances..
- Le graphe rend visibles couplage, cycles, dépendances critiques et SPOF..

# Partie 3 : Architecture, IoC et injection de dépendances ?

# L'architecture décide de vos dépendances

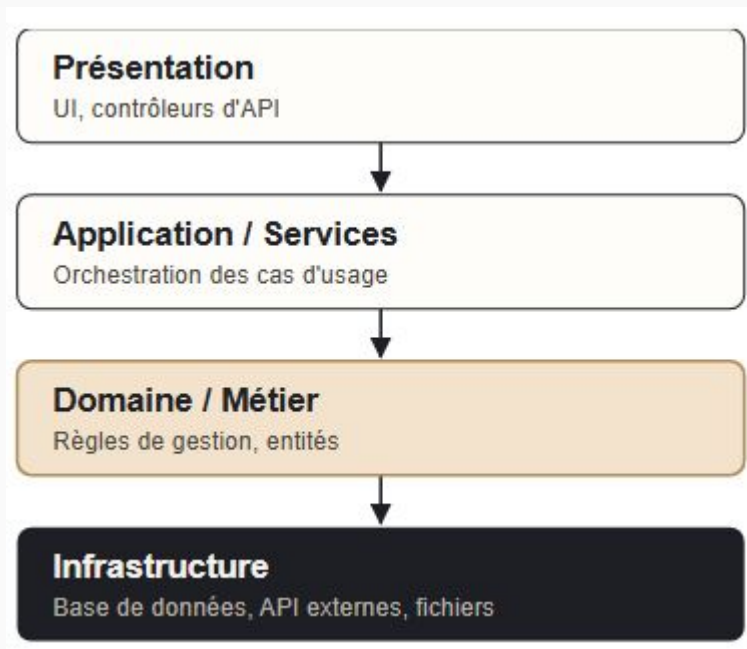
Selon l'organisation des modules, on peut :

- réduire ou augmenter le couplage
- faciliter ou compliquer la maintenance
- isoler ou propager les dépendances externes
- rendre le code testable... ou non
- limiter les effets domino d'un changement

**Une bonne architecture**  
**maîtrise les dépendances.**

**Une mauvaise architecture**  
**les subit.**

# L'architecture en couches



## Objectifs :

- Séparer les responsabilités
- Protéger le métier de la technique
- Limiter la propagation des dépendances externes
- Simplifier les tests
- Isoler base et API externes

Chaque couche dépend de celle du dessous, jamais de celle du dessus.

# Le problème du new partout

```
public class OrderService
{
    public void Confirm(Order order)
    {
        var email = new SmtplibEmailService();
        email.Send(order.CustomerEmail,
            "Commande confirmée");
    }
}
```

- **Fortement couplé** : Lié à SmtplibEmailService pour toujours.
- **Impossible à tester** : Chaque test envoie un vrai e-mail.
- **Fragile** : Si le constructeur change, l'appelant casse.
- **Rigide** : Passer au SMS impose de modifier la classe.

Le code choisit lui-même ses dépendances : c'est ce que l'IoC évite.

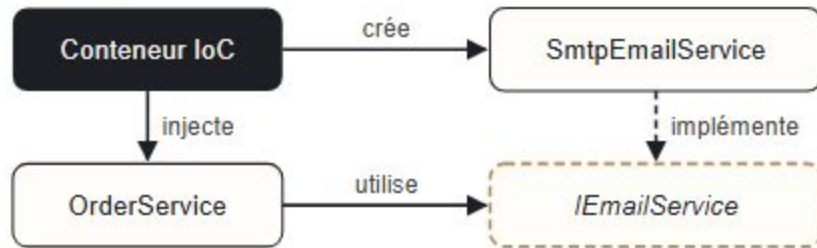
# Inversion de contrôle : ce n'est plus le module qui va chercher ses dépendances

## Sans IoC



Le service crée et choisit sa dépendance

## Avec IoC



- **Réutilisable** : Le module marche avec toute implémentation.
- **Testable** : On lui fournit un fake en test.
- **Remplaçable** : On change la dépendance sans toucher au module.

# L'injection de dépendances : trois formes

## RECOMMANDÉE

### Par constructeur

Claire, obligatoire, testable.

```
public OrderService(  
    IPaymentGateway gateway)  
{ _gateway = gateway; }
```

## OPTIONNELLE

### Par propriété

Moins stricte : peut rester vide.

```
public ILogger? Logger  
{ get; set; }
```

## PONCTUELLE

### Par méthode

Pour un besoin lié à un appel.

```
public void Export(  
    IExportFormat format)  
{ ... }
```

DI : le module ne crée plus ses dépendances, il les reçoit.

# Le conteneur IoC fait le travail

Ce que fait le conteneur :

- Il crée les objets
- Il gère leur durée de vie
- Il résout les dépendances en cascade
- Il permet de remplacer une implémentation en un point

|         |                                              |
|---------|----------------------------------------------|
| .NET    | Microsoft.Extensions.<br>DependencyInjection |
| Java    | Spring                                       |
| Node.js | InversifyJS                                  |
| Python  | dependency-injector                          |

# Durées de vie : Transient, Scoped, Singleton

|              |                          |                             |
|--------------|--------------------------|-----------------------------|
| AddTransient | à chaque résolution      | Services légers, sans état  |
| AddScoped    | par requête HTTP         | DbContext, unité de travail |
| AddSingleton | pour toute l'application | Cache, configuration        |

**Piège** : la dépendance captive

Un singleton qui reçoit un service scoped le garde pour toujours : état partagé entre requêtes.

# Pourquoi l'IoC rend le code testable

## Avant

```
var repo = new SqlUserRepository();  
var service = new UserService(repo);  
  
// Impossible de tester  
// sans base SQL disponible
```

## Avec IoC

```
class FakeUserRepository : IUserRepository  
{  
    public User? Find(string email)  
        ⇒ new User(email);  
}  
  
var service = new UserService(  
    new FakeUserRepository());  
Assert.True(service.Exists("a@b.fr"));
```

Tests unitaires simples - Aucune dépendance lourde - Aucun effet de bord - Tests rapides

# A retenir

- L'architecture en couches limite la propagation des dépendances.
- Le new dispersé crée un couplage fort et un code intestable.
- IoC : le module ne choisit plus ses dépendances. La DI les lui fournit.
- Le conteneur gère création et durée de vie ; les fakes rendent les tests simples.

# Partie 4 : TP1 : API Météo

# L'objectif

Construire une API qui reçoit une adresse postale en GET et renvoie les prévisions météo du lieu, avec les bonnes pratiques vues cet après-midi : couplage faible, IoC, DI.

Requête

```
GET /forecast?address=Alès
```

Réponse attendue

```
{  
  "address": "Alès",  
  "latitude": 44.13, "longitude": 4.08,  
  "hourly": { "shortwave_radiation": [ ... ] }  
}
```

# Deux services externes à enchaîner

- **GEOCODING : Nominatim (Nom de lieu → latitude / longitude)**

[nominatim.openstreetmap.org/search?q=Alès&format=json](https://nominatim.openstreetmap.org/search?q=Alès&format=json)

- **MÉTÉO : Open-Meteo (Latitude / longitude → prévisions)**

[api.open-meteo.com/v1/forecast?latitude=48.85&longitude=2.35&hourly=shortwave\\_radiation](https://api.open-meteo.com/v1/forecast?latitude=48.85&longitude=2.35&hourly=shortwave_radiation)

# Partie 5 : TP2 : API Météo

# Changement de sources

PDF

JOUR 2 — Découpler,  
packager et sécuriser ses  
choix.



# Terminer TP1 & TP2

- Construire une API GET adresse → prévisions, enchaînant géocodage et météo, avec couplage faible, IoC et DI — pas de new en dur sur les services externes.
- Faire évoluer l'architecture pour supporter plusieurs fournisseurs par abstraction, choisis sans recompilation, sans toucher au reste du code.
- Garantir, par des tests (unitaires, end-to-end, et de contrat par abstraction), que le code est fiable et qu'aucun DTO propre à un fournisseur ne fuit hors de son adaptateur.

# Partie 6 : Découpler ses dépendances



# La boundary (limite/frontière) : une frontière entre votre métier et le monde extérieur

## SANS BOUNDARY

```
public class TemperatureService
{
    public double GetTemperature()
    {
        var r = new HttpClient().GetAsync(
            "https://api.weather.com/today").Result;
        return double.Parse(
            r.Content.ReadAsStringAsync().Result);
    }
}
```

## AVEC BOUNDARY

```
public interface IWeatherProvider
{ double GetCurrentTemperature(); }

public class TemperatureService
{
    private readonly IWeatherProvider _provider;
    public double GetTemperature()
        ⇒ _provider.GetCurrentTemperature();
}
```

# Le seam (point de couture): un point où glisser un comportement différent

Un point d'insertion où l'on peut modifier le comportement d'un composant **sans toucher au reste du système**. La boundary crée le seam ; le seam est ce qu'on utilise concrètement en test.

Fake pour test, grâce au seam (l'interface)

```
public class FakeWeatherProvider : IWeatherProvider
{ public double GetCurrentTemperature() => 25; }

var service = new TemperatureService(new FakeWeatherProvider());
Assert.Equal(25, service.GetTemperature());
```

# Exemple d'un module fortement couplé

```
public class UserService
{
    public void Register(string email)
    {
        // 1. Logique métier mélangée avec de la technique
        var user = new User(email);

        // 2. Appel direct à une implémentation concrète
        var smtp = new SmtpClient("smtp.exemple.com");
        smtp.Send("noreply@exemple.com", email,
            "Bienvenue !", "Merci pour votre inscription");

        // 3. Pas de point d'injection → impossible à mocker
        Console.WriteLine("Email envoyé !");
    }
}
```

# Le même module, faiblement couplé

```
public interface IEmailService
{ void SendWelcomeEmail(string email); }

public class SmtplibEmailService : IEmailService
{
    public void SendWelcomeEmail(string to)
    {
        var smtp = new SmtplibClient("smtp.exemple.com");
        smtp.Send("noreply@exemple.com", to,
            "Bienvenue !", "Merci !");
    }
}
```

```
public class UserService
{
    private readonly IEmailService _email;

    public UserService(IEmailService email)
    { _email = email; }

    public void Register(string email)
    {
        var user = new User(email);
        _email.SendWelcomeEmail(email); // pas de technique ici
    }
}
```

# Qu'est-ce qu'un design pattern ?

Une solution **nommée et réutilisable** à un problème de conception qui revient souvent. Pas du code à copier-coller : une structure à adapter à votre contexte.

## D'où ça vient

Catalogué par le "Gang of Four" (Gamma, Helm, Johnson, Vlissides) en 1994 — 23 patterns classés en création, structure, comportement.

## Pourquoi ça compte ici

Un vocabulaire commun : dire "Adapter" à un collègue ou à une IA évite de réexpliquer toute la structure à chaque fois.

## Un piège à éviter

Un pattern n'est pas un objectif en soi. On l'utilise pour résoudre un problème de couplage réel, jamais pour "faire propre".

# Quatre patterns, quatre problèmes différents

**Adapter** : L'interface externe ne convient pas à mon domaine

**Facade** : Trop de dépendances techniques à orchestrer

**Strategy** : Plusieurs comportements interchangeables

**Factory** : La création d'un objet est trop complexe

# Adapter · un bouclier contre les dépendances externes

Rendre compatible un SDK ou une API externe avec le modèle interne, sans que le domaine connaisse cette API.

- API externe qui renvoie un format différent
- Librairie dont l'interface ne convient pas
- Besoin d'isoler les changements externes

```
public interface IEmailPort
{ void Send(string to, string msg); }

// SDK externe très différent
public class ExternalEmailSdk
{ public void SendMail(string addr, string body, bool hi)
{} }

public class ExternalEmailAdapter : IEmailPort
{
    private readonly ExternalEmailSdk _sdk = new();
    public void Send(string to, string msg)
        ⇒ _sdk.SendMail(to, msg, false);
}
```

# Facade · masquer la complexité

Simplifier l'accès à un sous-système qui demande d'orchestrer plusieurs dépendances techniques dans le bon ordre.

- Regrouper plusieurs appels techniques sous une seule méthode
- Simplifier une API interne difficile à utiliser
- Fournir une interface claire, décorrélée de la technique

```
public class UserAccountFacade
{
    private readonly IUserRepository _repo;
    private readonly IPasswordHasher _hasher;
    private readonly IEmailService _email;

    // ... constructeur avec les 3 dépendances

    public void CreateUser(string email, string pwd)
    {
        var hashed = _hasher.Hash(pwd);
        _repo.Save(new User(email, hashed));
        _email.SendWelcomeEmail(email);
    }
}
```

# Strategy · changer un comportement sans changer l'appelant

Plusieurs façons de faire la même chose, choisies au runtime.

- Plusieurs façons de calculer un prix
- Plusieurs méthodes d'authentification
- Le mode démo du TP3 : vraie API ou données fake

Le Strategy diminue le couplage au comportement.

```
public interface IWeatherService
{ Task<Forecast> GetAsync(Coord c); }

public class OpenMeteoWeatherService
    : IWeatherService { /* vrai appel */ }

public class DemoWeatherService
    : IWeatherService { /* données fake */ }

// dans Program.cs, selon demo=true
services.AddScoped<IWeatherService>(sp =>
    isDemo ? new DemoWeatherService()
           : sp.GetRequiredService<OpenMeteoWeatherService>());
```

# Factory · centraliser une création complexe

Utile quand la construction est compliquée, que plusieurs dépendances doivent être fournies, ou que l'objet créé dépend du contexte.

- Création isolée
- Pas de new éparpillé partout

IoC <> Factory mais injecté dans la DI

```
public interface IPaymentProcessor
{ void Pay(decimal amount); }

public static class PaymentFactory
{
    public static IPaymentProcessor Create(string type)
    {
        ⇒ type switch
        {
            "card" ⇒ new CardPaymentProcessor(),
            "paypal" ⇒ new PaypalPaymentProcessor(),
            _ ⇒ throw new ArgumentException("Type inconnu")
        };
    }
}
```

# A retenir

- La boundary isole le métier stable de la technique instable.
- Le seam est le point où l'on glisse un fake pour tester.
- Adapter isole une API externe
- Facade orchestre plusieurs dépendances.
- Strategy choisit un comportement
- Factory centralise une création complexe.

# Partie 7 : TP3 : API Météo

# Ajout de fonctionnalités

**TP3.PDF disponible dans Campus**

# Partie 8 : Packages et licences



# Un package, un gestionnaire, une chaîne

## **Qu'est-ce qu'un package ?**

Un morceau de code réutilisable, publié par quelqu'un d'autre : bibliothèque, framework, wrapper d'API, SDK cloud, outil d'intégration.

## **Le package manager**

Télécharge, installe, gère les versions et les dépendances transitives, met à jour.  
NPM, NuGet, pip/Poetry.

**Installer un package = ajouter une dépendance externe, et toute sa chaîne transitive.**

## Trois familles de licences

- **PERMISSIVE**
- **COPYLEFT**
- **PROPRIÉTAIRE**

## Permissive

Les licences permissives sont les licences open-source les plus simples et les moins contraignantes.

Elles autorisent presque tout, y compris l'usage commercial, sans obligation d'ouvrir votre propre code.

MIT : Massachusetts Institute of Technology en 1998

Apache 2.0 : Apache Software Foundation (ASF) en 2004

BSD : Berkeley Software Distribution en 1988

Si vous utilisez ou modifiez un code sous Copyleft, vous devez publier vos modifications — parfois même tout votre code source.

Elles protègent la liberté du logiciel... mais au prix de contraintes légales élevées, souvent incompatibles avec les projets propriétaires.

GPLv2 : GNU General Public License en 1991

GPLv3 : GNU General Public License en 2007

AGPLv3 : Affero General Public License en 2007

LGPLv2.1: GNU Lesser General Public License en 1999

LGPLv3: GNU Lesser General Public License en 2007

Toutes par Free Software Foundation (FSF).

Les licences propriétaires – aussi appelées closed source – sont des licences où le code source n'est pas librement réutilisable, et où l'éditeur impose des conditions strictes d'utilisation.

Vous n'avez pas de droits sur le code, seulement une autorisation d'utilisation définie par le contrat.

Elles sont fréquentes dans les environnements professionnels, dans certains SDK, clients API, bibliothèques de chiffrement, connecteurs, etc.

## Toujours vérifier la licence + les dépendances transitives

Quand on installe un package (NuGet, NPM, PIP), on n'installe pas seulement une dépendance.

On installe toute sa chaîne de dépendances, parfois des dizaines ou centaines d'autres packages.



Vous êtes légalement responsables de TOUTES les licences présentes dans la chaîne.

# Versionning

Le versionning permet de comprendre l'impact d'une mise à jour de package

Un numéro de version = un niveau de risque + un niveau de compatibilité

MAJOR.MINOR.PATCH : Exemple : 2.4.7

## **MAJOR (x.0.0)**

Rupture de compatibilité  
Modifications importantes de l'API  
Peut casser votre code  
Nécessite audit + tests complets

1.9.4 → 2.0.0

0.x → 1.0.0

## **MINOR (1.x.0)**

Nouvelle fonctionnalité  
Pas de rupture (en théorie)  
Faible risque, mais doit être testé

2.3.1 → 2.4.0

## **PATCH (1.0.x)**

Corrections de bugs  
Pas de nouvelles fonctionnalités  
Pas de rupture  
Mise à jour généralement sûre

2.4.7 → 2.4.8

# A retenir

- Un package installé, c'est toute une chaîne transitive de licences.
- Permissive, copyleft, propriétaire : chacune a ses obligations légales.
- Un numéro de version indique le niveau de risque d'une mise à jour.

# Partie 9 : Auditer les licences de votre projet



# À vous : scannez votre propre projet

Lire les manifestes à la main ne passe pas à l'échelle sur des centaines de packages transitifs. Installez et lancez l'outil de votre écosystème sur le code du TP1/TP2/TP3 et ou vos projets fil rouge.

**TP4.PDF disponible dans Campus**

# Partie 10 : Dépendances stratégiques



# Vendor lock-in : sortir coûte cher

- **Services managés propriétaires** : Fonctions serverless, bases de données propriétaires, IA maison
- **Formats fermés** : Données exportables uniquement dans un format propriétaire
- **Frais de sortie** : Facturation de l'egress, des données qui sortent du cloud
- **Compétences spécifiques** : L'équipe ne sait travailler qu'avec cet écosystème

**La boundary et l'Adapter sont aussi des outils anti-lock-in.**

# Debrief · combien vous a coûté le changement de fournisseur ?

Le TP2 vous a demandé de remplacer Nominatim par la BAN et Open-Meteo par MET Norway. C'était un lock-in miniature, mesuré en heures plutôt qu'en années.

- **Fichiers touchés** : 2 nouveaux adaptateurs seulement ?
- **Tests cassés** : 0, ou fallait-il tout réécrire ?
- **Temps passé** : 20 min ou 2h ?
- **Fuites de DTO** : Un champ Nominatim traînait-il ailleurs ?

**Le coût que vous venez de vivre EST la mesure du lock-in. Un vrai fournisseur cloud, ce sont les mêmes questions à l'échelle d'une migration de plusieurs mois.**

# Pricing : le fournisseur peut changer les règles seul

| Année | Événement                                                   |
|-------|-------------------------------------------------------------|
| 2021  | Elastic passe sous SSPL → fork OpenSearch par la communauté |
| 2022  | Heroku supprime son offre gratuite                          |
| 2023  | L'API Twitter/X devient payante                             |
| 2023  | Unity annonce une "runtime fee", puis recule partiellement  |
| 2023  | HashiCorp passe Terraform sous BSL → fork OpenTofu          |
| 2024  | Redis change de licence → fork Valkey                       |
| 2024  | Broadcom refond les licences VMware après le rachat         |

# Souveraineté : quel droit s'applique à vos données ?

- **Cloud Act** : Extraterritorialité du droit américain sur les opérateurs US, même hébergés en Europe
- **RGPD, Schrems II et DPF** : Privacy Shield invalidé en 2020, remplacé par le DPF en 2023, validé en 1re instance en 2025 mais contesté en appel devant la CJUE. Un cadre de transfert peut tomber : prévoir un plan de sortie.
- **SecNumCloud / HDS** : Qualification ANSSI et hébergement santé, gages de souveraineté en France

**Localiser la donnée ne suffit pas : la nationalité de l'opérateur compte aussi.**

# Pérennité et dépendance humaine

- **Le fournisseur** : Peut faire faillite, être racheté, ou mettre un produit en fin de vie (end of life) du jour au lendemain.
- **Le projet open source** : Peut n'avoir qu'un seul mainteneur actif : le "bus factor" du projet lui-même.
- **Votre équipe** : Une compétence rare peut reposer sur une seule personne en interne.

# Le SBOM : la liste des ingrédients de votre logiciel

-